K L UNIVERSITY

FRESHMAN ENGINEERING DEPARTMENT

A Project Based Lab ReportOn

MINI ARCADE GAME SUIT

SUBMITTED BY:

ID NUMBER	NAME
2500030040	P. Lakshmi
2500090020	D. Hema Hasitha
2500090082	Sai Sruti
2500090248	K. Nikhil

Under the Esteemed Guidance Of

Dr. E. SRIDEVI

PROFESSOR, C.S.E

KL UNIVERSITY

Green fields, Vaddeswaram – 522
502Guntur Dt., AP, India.

DEPARTMENT OF BASIC ENGINEERING SCIENCE-1

CERTIFICATE

This is to certify that the project based laboratory report entitled "MINI ARCADE GAME SUIT" submitted by **Lakshmi, Hema Hasitha, Sai Sruti, Nikhil** bearing Regd. No. 2500030040, 2500090020, 2500090082, 2500090248 to the **Department of COMPUTER SICENCE ENGINEERING, KL University** in partial fulfillment of the requirements for the completion of a project based Laboratory in "**Python programming (25SC2007)"** course in I B Tech II Semester, is a Bonafede record of the work carried out by them under my supervision during the academic year 2025-2026.

Dr E SREEDEVI	Dr. D. HARITHA
PROJECT SUPERVISOR	HEAD OF THE DEPARTMENT

ACKNOWLEDGEMENTS

It is great pleasure for me to express my gratitude to our honorable President. **Sri. Koneru Satyanarayana**, for giving the opportunity and platform with facilities in accomplishing the project-based laboratory report.

We express sincere gratitude to our Director **Dr. A. Jagdeesh** for his administration towards our academic growth. We express sincere gratitude to our Principal **Dr. V. Krishna Reddy** for his administration towards our academic growth.

We express sincere gratitude to our HOD-BES-1 **Dr. D. Haritha** for her leadership and constant motivation provided for the successful completion of our academic semester. We record it as my privilege to deeply thank you for providing us with the efficient faculty and facilities to make our ideas into reality.

We express our sincere thanks to our project supervisor **Dr E Sreedevi** for her novel association of ideas, encouragement, appreciation, and intellectual zeal which motivated us to venture this project successfully.

Finally, we are pleased to acknowledge my indebtedness to all those who devoted themselves directly orindirectly to making this project a success.

Project Associates…

Lakshmi-2500030040

Hema hasitha-2500090020

Sai sruti-2500090082

Nikhil-2500090248

ABSTRACT

The Mini Arcade Game Suite is a software application designed to provide a collection of simple and interactive games within a single platform. The main objective of this project is to offer entertainment while demonstrating fundamental programming concepts such as control structures, functions, data handling, and user interaction.

This game suite typically includes multiple mini-games such as number guessing, tic-tac-toe, rock-paper-scissors, and memory-based challenges. Each game is designed with a user-friendly interface, allowing players to easily navigate between different options through a menu-driven system. The program ensures smooth gameplay by handling inputs efficiently and providing instant feedback to users.

Key Words: Mini Arcade, Game Suite, Interactive Games, Menu-Driven, Program User Interface, Game Development

INDEX

S.NO	TITLE	PAGE NO
1	Introduction	1-4
2	Aim of the Project	5
2.1	Advantages & Disadvantages	6
2.2	Future Implementation	6
3	Software & Hardware Details	7
4	Data Flow Diagram	8-9
5	Algorithm for each module	10-13
6	Implementation	14-18
7	Integration and System Testing	19-20
8	Visualization	21
9	Conclusion	22

INTRODUCTION

The Mini Arcade Game Suite is a simple and fun project that brings together multiple small games into one application. Instead of playing different games separately, this project allows users to access and enjoy a variety of games in a single place through an easy menu system.

The main idea behind this project is to combine entertainment with learning. It helps in understanding basic programming concepts like loops, conditions, and functions in a practical way. Each game in the suite is designed to be simple, interactive, and user-friendly, so that anyone can play without difficulty.

This project is especially useful for beginners, as it improves logical thinking and problem-solving skills while keeping the experience enjoyable. Overall, the Mini Arcade — Game Suite is a creative way to learn programming by building and playing multiple games in one platform.

The Mini Arcade Game Suite (MAGS) is a simple yet interactive console-based gaming application developed using the Python programming language. The main purpose of this project is to provide a platform where users can play multiple games within a single system in an easy and engaging manner.

This project combines basic games such as the Number Guessing Game and Hangman Game, which are designed to improve the user's logical thinking, decision-making skills, and problem-solving abilities. The system follows a menu-driven approach, allowing users to select their desired game, view their performance, or exit the application smoothly.

One of the key features of this project is the implementation of a player management system, which keeps track of important details such as the number of games played, wins, and losses, thereby making the application more interactive and personalized. The project is developed using modular programming, where different functionalities are divided into separate modules such as game logic, player management, menu handling, and main control, making the code more organized, readable, and easy to maintain. Additionally, object-oriented

programming concepts are used to manage player data efficiently through the use of classes and objects.

This project not only demonstrates the practical application of fundamental programming concepts like loops, conditional statements, functions, and data structures, but also provides a strong foundation for The Mini Arcade Game Suite is a simple and interactive project that brings multiple games into one application. Instead of playing different games separately, this system allows users to access and enjoy a variety of games in a single place through a menu-driven interface. This makes the application convenient, user-friendly, and easy to navigate for all types of users.

The main idea behind this project is to combine entertainment with learning. It helps in understanding basic programming concepts like loops, conditional statements, and functions in a practical way. Each game in the suite is designed to be simple, interactive, and easy to understand, so that users can play without difficulty and also learn simultaneously.

This project includes games such as the Number Guessing Game and Hangman Game, which focus on improving logical thinking, decision-making, and problem-solving skills. The system also keeps track of the player's performance, including the number of games played, wins, and losses, making it more engaging and personalized for the user.

The project is developed using modular programming, where different parts of the program are divided into separate modules such as game logic, player management, and menu control. This structure makes the program well-organized, easy to read, and simple to maintain. It also uses object-oriented programming concepts to manage player data efficiently.

This project is especially useful for beginners, as it enhances coding skills while keeping the experience enjoyable. Overall, the Mini Arcade Game Suite is a creative and effective way to learn programming by building and playing multiple games within a single platform. building more advanced gaming systems. Furthermore, the design of the Mini Arcade Game Suite is scalable, meaning that more games and features can be added in the future without

affecting the existing structure, making it a flexible and extensible system. Overall, this project serves as a great learning tool for beginners and showcases how basic programming knowledge can be applied to develop a functional and enjoyable software application.

The Modules in our project are: -

1. Game Module: Guessing Game

- **Purpose:** A simple number guessing challenge where the player tries to guess a randomly generated number.
- **Key Features:**
  - Uses the random module to generate a number between 1 and 100.
  - Tracks the number of attempts.
  - Provides feedback ("Too low", "Too high") until the correct guess is made.
  - Updates the player's win record when successful.
- **Learning Concepts:** Loops, conditionals, user input handling, randomization.

2. Game Module: Hangman Game

- **Purpose:** A word guessing game where the player guesses letters to reveal a hidden word.
- **Key Features:**
  - Maintains a list of possible words and selects one randomly.
  - Tracks guessed letters and remaining attempts.
  - Displays the word with blanks for unguessed letters.
  - Updates win/loss stats depending on the outcome.
- **Learning Concepts:** Sets, string manipulation, loops, conditionals, user interaction.

3. Player Module

- **Purpose:** Represents the player and keeps track of their performance across games.
- **Key Features:**
  - Stores player name, games played, wins, and losses.
  - Provides methods to update wins/losses.
  - Displays overall statistics.
- **Learning Concepts:** Object-Oriented Programming (OOP), class design,

encapsulation.

4. Menu Module

- **Purpose:** Provides a central menu for the player to choose which game to play or view stats.
- **Key Features:**
  - Displays options (Guessing Game, Hangman Game, View Stats, Exit).
  - Acts as a navigation hub for the suite.
- **Learning Concepts:** Modular design, user interface (text-based), function calls.

AIM

The aim of the Mini Arcade — Game Suite project is to develop a simple and interactive application that combines multiple mini-games into a single platform, providing entertainment while improving programming and problem-solving skills.

It also aims to help beginners understand fundamental concepts like control structures, functions, and user interaction through practical implementation in game development.

ADVANTAGES

- Provides entertainment through multiple games in one place
- Helps in learning programming concepts (loops, conditions, functions)
- Improves logical thinking and problem-solving skills
- Easy to use with a menu-driven interface
- Saves time by combining many games in a single application

DISADVANTAGES

- Limited to simple games (not advanced graphics or features)
- May become boring over time without updates
- Lacks high-quality graphics and sound effects
- Performance depends on coding efficiency

FUTURE IMPLEMENTATION

- Limited to simple games (not advanced graphics or features)
- May become boring over time without updates
- Lacks high-quality graphics and sound effects
- Performance depends on coding efficiency

SYSTEM REQUIREMENTS

➢ SOFTWARE REQUIREMENTS:

The major software requirements of the project are as follows:Language: Turbo-C.
Operating system**:** Windows 10 or more.
Technical requirements: Monitor, CPU, Keyboard, Mouse, etc...

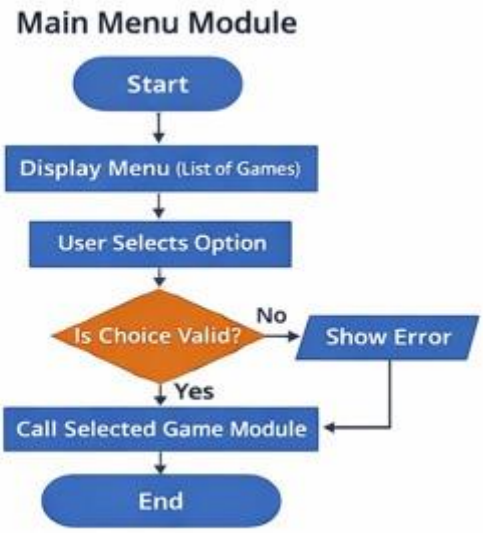
➢ HARDWARE REQUIREMENTS:

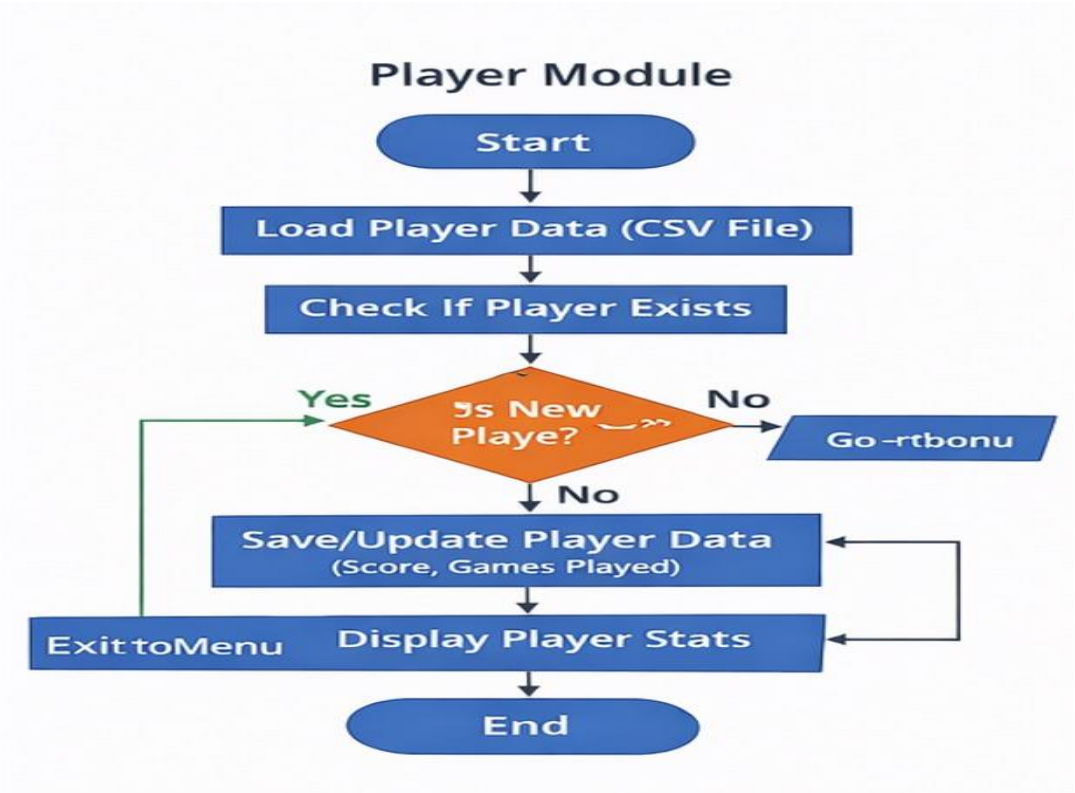
The hardware requirements that map towards the software are as

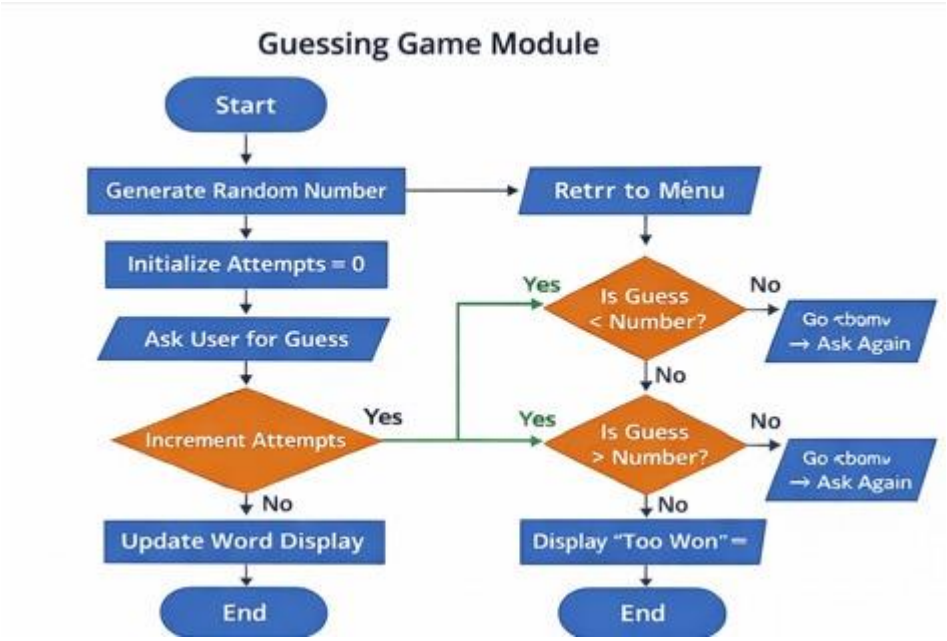
follows:RAM: 8 GB.

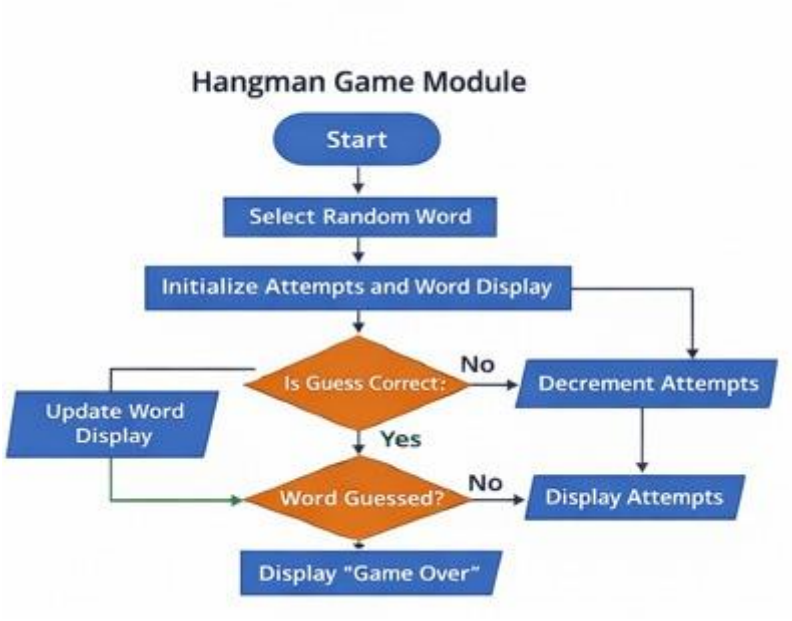
Processor: i5/i7.

DATA FLOW DIAGRAM






Guessing Game Module
Start
Generate Random Number
Retrr to Menu
Initialize Attempts = 0
Ask User for Guess
Yes
Is Guess < Number?
No
Go <bomv → Ask Again
No
Increment Attempts
Yes
Yes
Is Guess > Number?
No
Go <bomv → Ask Again
No
No
Update Word Display
Display "Too Won" =
End
End


Hangman Game Module
Start
Select Random Word
Initialize Attempts and Word Display
Is Guess Correct:
No
Decrement Attempts
Update Word Display
Yes
Word Guessed?
No
Display Attempts
Display "Game Over"

ALGORITHM

1. Algorithm: Game Module (Guessing Game)

Function: guessing_game(player)

1. Start
2. Generate a random number between 1 and 100
3. Initialize attempts = 0
4. Display message to guess number
5. Repeat:
    - Take user input (guess)
    - Increment attempts
    - If guess < number → print "Too low"
    - Else if guess > number → print "Too high"
    - Else:
        - Print "Correct!"
        - Call player.update_win()
        - Exit loop
6. Increment player.games_played
7. End

---

2. Algorithm: Game Module (Hangman Game)

Function: hangman_game(player)

1. Start
2. Create list of words
3. Select a random word
4. Initialize:
    - guessed = empty set
    - attempts = 6
5. Repeat while attempts > 0:

- Display word with blanks
- Take user input (letter)
- If letter is in word:
  - Add to guessed set
- Else:
  - Decrease attempts
  - Display remaining attempts
- If all letters guessed:
  - Print “You won!”
  - Call player.update_win()
  - Exit loop

6. If attempts == 0:
   - Print “You lost”
   - Call player.update_loss()
7. Increment player.games_played
8. End

---

3. Algorithm: Player Module

Class: Player

Initialization (__init__)

1. Start
2. Take player name
3. Set:
   - games_played = 0
   - wins = 0
   - losses = 0
4. End

Update Win (update_win)

1. Start
2. Increment wins by 1
3. End

Update Loss (update_loss)

1. Start
2. Increment losses by 1
3. End

Display Stats (display_stats)

1. Start
2. Print player name
3. Print games played
4. Print wins
5. Print losses
6. End

4. Algorithm: Menu Module

Function: display_menu()

1. Start
2. Display menu options:
    - Guessing Game
    - Hangman Game
    - View Stats
    - Exit
3. End

5. Algorithm: Main Control Module

Function: main()

1. Start
2. Ask user to enter name
3. Create Player object
4. Repeat:
    - Display menu
    - Take user choice
    - If choice = 1:

        - Call guessing_game(player)
    - Else if choice = 2:
        - Call hangman_game(player)
    - Else if choice = 3:
        - Call display_stats()
    - Else if choice = 4:
        - Print exit message
        - Stop program
    - Else:
        - Print “Invalid choice”
5. End

IMPLEMENTATION

1.payer.py

```
class Player:
    def __init__(self, username):
        self.username = username
        self.games_played = 0
        self.wins = 0
        self.losses = 0

    def update_win(self):
        self.wins += 1

    def update_loss(self):
        self.losses += 1

    def show_stats(self):
        print("\n--- Stats ---")
        print("Username:", self.username)
        print("Games Played:", self.games_played)
        print("Wins:", self.wins)
        print("Losses:", self.losses)

    def to_dict(self):
        return self.__dict__

    @staticmethod
    def from_dict(data):
        p = Player(data["username"])
        p.games_played = data["games_played"]
        p.wins = data["wins"]
        p.losses = data["losses"]
        return p
```

2.games.py

```
import random

def guessing_game(player):
    number = random.randint(1, 100)
    attempts = 0

    print("\nGuess a number between 1 and 100")
```

```
    while True:
        guess = int(input("Enter guess: "))
        attempts += 1

        if guess < number:
            print("Too low!")
        elif guess > number:
            print("Too high!")
        else:
            print("Correct!")
            player.update_win()
            break

    player.games_played += 1


def hangman_game(player):
    words = ["python", "arcade", "game", "code"]
    word = random.choice(words)
    guessed = set()
    attempts = 6

    print("\nHANGMAN GAME")

    while attempts > 0:
        display = "".join([c if c in guessed else "_" for c in word])
        print(display)

        if "_" not in display:
            print("You won!")
            player.update_win()
            break

        guess = input("Enter letter: ")

        if guess in word:
            guessed.add(guess)
        else:
            attempts -= 1
            print("Wrong! Attempts left:", attempts)

    if attempts == 0:
        print("You lost! Word was:", word)
        player.update_loss()

    player.games_played +=1
```

3.storage.py

```
import csv
import os
from player import Player

FILE = "players.csv"

# Create file if not exists
if not os.path.exists(FILE):
    with open(FILE, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["username", "games_played", "wins", "losses"])


def save_profile(player):
    rows = []
    found = False

    # Read existing data
    with open(FILE, "r") as f:
        reader = csv.DictReader(f)
        for row in reader:
            if row["username"] == player.username:
                row = player.to_dict()
                found = True
            rows.append(row)

    # Add new player if not found
    if not found:
        rows.append(player.to_dict())

    # Write back to CSV
    with open(FILE, "w", newline="") as f:
        writer = csv.DictWriter(
            f, fieldnames=["username", "games_played", "wins", "losses"]
        )
        writer.writeheader()
        writer.writerows(rows)


def load_profile(username):
    if not os.path.exists(FILE):
        return None
```

```
    with open(FILE, "r") as f:
        reader = csv.DictReader(f)
        for row in reader:
            if row["username"] == username:
                row["games_played"] = int(row["games_played"])
                row["wins"] = int(row["wins"])
                row["losses"] = int(row["losses"])
                return Player.from_dict(row)

    return None
```

4.main.py

```
from player import Player
from games import guessing_game, hangman_game
from storage import save_profile, load_profile


def main():
    print(" Welcome to Mini Game Hub ")

    username = input("Enter username: ")

    # Load player from CSV
    player = load_profile(username)

    if player is None:
        print("New player created!")
        player = Player(username)
    else:
        print(f"Welcome back, {player.username} ")

    while True:
        print("\n----- MENU -----")
        print("1. Guessing Game")
        print("2. Hangman Game")
        print("3. Show Stats")
        print("4. Exit")

        choice = input("Enter choice: ")

        if choice == "1":
            guessing_game(player)

        elif choice == "2":
            hangman_game(player)
```

```
        elif choice == "3":
            player.show_stats()

        elif choice == "4":
            save_profile(player)
            print(" Profile saved in players.csv")
            print(" Goodbye!")
            break

        else:
            print(" Invalid choice, try again!")


if __name__ == "__main__":
    main()
```

INTEGRATION AND SYSTEM TESTING OUTPUTS

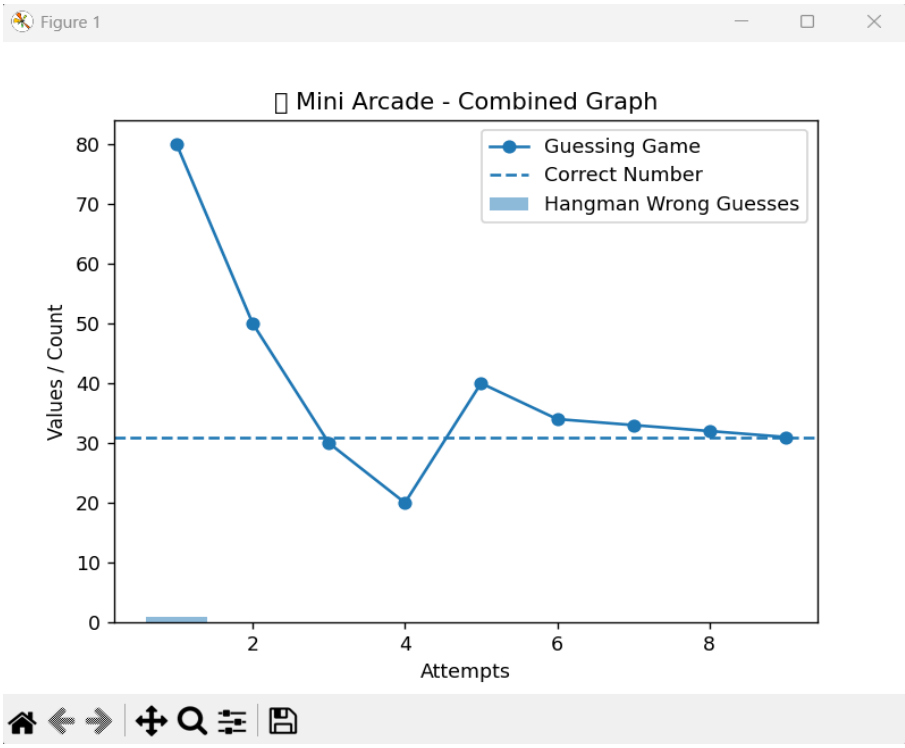
Screen Shots:

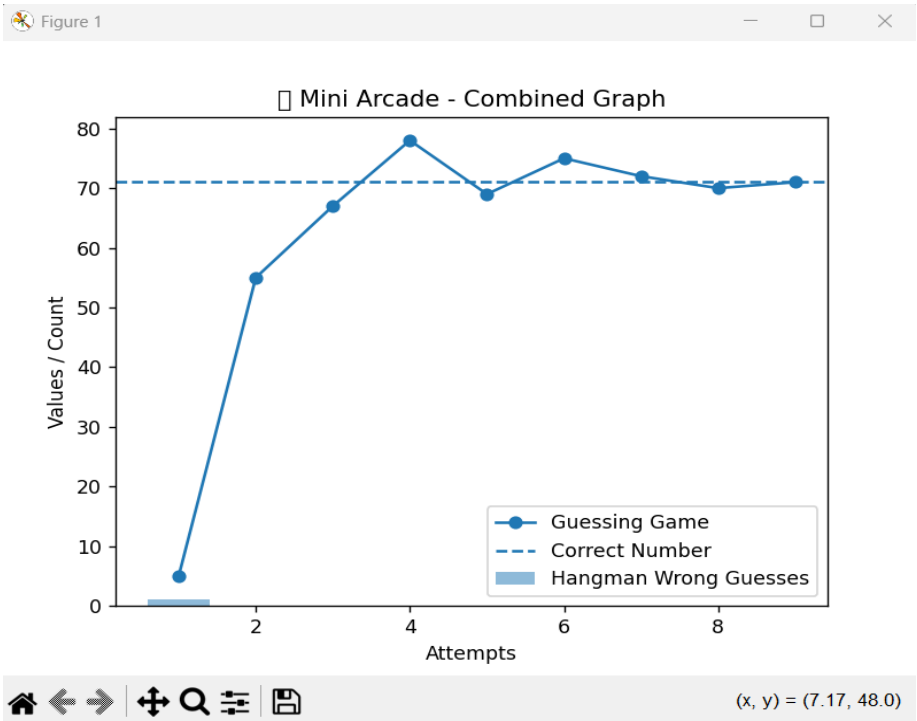
```
Guess a number between 1 and 100
Enter guess: 35
Too low!
Enter guess: 60
Too low!
Enter guess: 70
Too low!
Enter guess: 90
Too high!
Enter guess: 85
Too high!
Enter guess: 80
Too high!
Enter guess: 78
Too high!
Enter guess: 77
Too high!
Enter guess: 74
Too high!
Enter guess: 73
Correct!

--- MINI ARCADE ---
1. Guessing Game
2. Hangman
3. Show Stats
4. Exit
Enter choice:
```

```
Enter your username: nikhil

--- MINI ARCADE ---
1. Guessing Game
2. Hangman
3. Show Stats
4. Exit
Enter choice: 2

HANGMAN GAME
____
Enter letter: e
___e
Enter letter: g
g__e
Enter letter: a
ga_e
Enter letter: m
game
You won!

--- MINI ARCADE ---
1. Guessing Game
2. Hangman
3. Show Stats
4. Exit
Enter choice:
```

```
--- MINI ARCADE ---
1. Guessing Game
2. Hangman
3. Show Stats
4. Exit
Enter choice: 3

--- Stats ---
Username: nikhil
Games Played: 1
Wins: 1
Losses: 0

--- MINI ARCADE ---
1. Guessing Game
2. Hangman
3. Show Stats
4. Exit
Enter choice: 4
Goodbye!

Process finished with exit code 0
```

VISUALIZATION





CONCLUSION

The Mini Arcade Game Suite project successfully combines multiple simple games into a single, user-friendly application. It serves as both an entertaining platform and a practical learning tool for understanding basic programming concepts like loops, conditions, and functions. Through this project, users can improve their logical thinking and problem-solving skills while enjoying interactive gameplay. Although the current version includes basic features, it provides a strong foundation for further enhancements such as better graphics, additional games, and advanced functionalities. Overall, the Mini Arcade — Game Suite is an effective and creative project that balances learning and entertainment, making it especially useful for beginners in programming.